Building an accessibility app for visual impairment that calculates distance and identifies objects requires a specific setup in **Cursor** to handle high-performance tasks like live camera analysis and text-to-speech.

Since mobile hardware varies, the most reliable way to calculate distance without specialized LiDAR sensors is through **Monocular Depth Estimation** (using known object heights and focal lengths) or **Fiducial Markers** (like QR-like tags with known sizes).

Phase 1: The Master "System Prompt" for Cursor

To get the best results, open **Cursor Composer (Ctrl + I)** and use this detailed prompt. It provides the "technical scaffolding" the AI needs to build a complex vision app.

> **Prompt:** "Build a mobile app using **React Native and Expo**. The app is an accessibility tool for the visually impaired.
>
> 1. **Core Feature:** Real-time object detection using a lightweight model (like TensorFlow Lite or MediaPipe).
> 2. **Distance Logic:** Implement a function that estimates distance based on the bounding box height of detected objects (assume standard heights for common objects like 'chair' or 'person').
> 3. **Baseline/Orientation:** Use the device's accelerometer/gyroscope to determine the 'baseline' (the floor level) to help orient the user.
> 4. **Accessibility:** Integrate **Text-to-Speech (TTS)** to announce: '[Object Name] detected at [Distance] meters, [Direction]'.
> 5. **UI:** A full-screen camera preview with a simple 'tap to scan' or continuous 'auto-scan' mode. Ensure high-contrast colors for the remaining UI elements."

Phase 2: What You Need to Do (Step-by-Step)

To finish this project on time, follow this specific order of operations in your **Cursor** environment:

1. **Initialize with Expo:** Open your terminal in Cursor and run npx create-expo-app@latest vision-aid. This creates the standard mobile framework.
2. **Install Essential Libraries:** You will need specific "packages" to handle the camera and audio. Tell Cursor to run:
    - npx expo install expo-camera expo-sensors expo-speech
    - npm install @tensorflow/tfjs @tensorflow-models/coco-ssd (for the object detection).
3. **The "Transfer" Strategy:**
    - Once Cursor generates the code, the files live on your computer as a standard

project.
    - To "change this into a normal mobile app," you can simply zip this folder or push it to **GitHub**.
    - **To Compile:** Use the **Expo Application Services (EAS)**. Run eas build -p android --profile preview to generate an **APK** file that you can install on any Android phone immediately.

Phase 3: Technical Constraints to Watch

- **Distance Accuracy:** Without LiDAR, distance is an estimate. The AI will use the **Pin-hole Camera Model** formula: $Distance = \frac{RealHeight \times FocalLength}{PixelHeight}$.
- **Performance:** Real-time detection is heavy on the battery. Ask Cursor to "Optimize the frame rate to 5–10 FPS" to keep the app from crashing on older phones.
- **Audio Feedback:** Ensure the app doesn't "shout" too many objects at once. Ask Cursor to "Implement a 3-second throttle on the Text-to-Speech announcements".

Would you like me to generate the specific "Distance Calculation" function in JavaScript that you can copy-paste into Cursor right now?